

```
In [ ]: with rasterio.open(raster_files[0]) as src:
        # Read the data and mask it with the shapefile geometry
        # get crs
        crs = src.crs
        print(f'CRS: {crs}')

        soc_utm = soilpoints.to_crs(crs.to_epsg())
```

CRS: EPSG:32636

```
In [67]: def extract_raster_values(point, raster_path):
        with rasterio.open(raster_path) as src:
            try:
                value = next(src.sample([(point.x, point.y)]))[0]
                return value
            except IndexError:
                return np.nan # Handle cases where point might be outside raster extent

        for raster_file in raster_files:
            get_first_name = lambda path: os.path.splitext(os.path.basename(path))[0]
            column_name = get_first_name(raster_file)
            soc_utm[column_name] = soc_utm.apply(lambda row: extract_raster_values(row.geometry, raster_file), axis=1)

        # If your SOTERv2_Sichei is a raster with categorical soil types:
        # You might need to extract the soil type at each point as well.
        # Then, you'll likely need to handle categorical variables (e.g., one-hot encoding)
        # If it's a vector layer, you'd do a spatial join.

        soc_utm.head()
```

```
Out[67]:
```

	geometry	B11	B12	B2	B3	B4	B5	B6
	POINT							
0	(682569.883 78665.874)	0.290867	0.197833	0.048367	0.081767	0.088233	0.136800	0.224033
	POINT							
1	(682539.797 78914.194)	0.323333	0.249733	0.049567	0.078367	0.094800	0.159800	0.224500
	POINT							
2	(682559.703 79172.464)	0.288000	0.217133	0.054700	0.087900	0.095100	0.136300	0.239200
	POINT							
3	(682669.568 79530.099)	0.284400	0.188667	0.040300	0.069233	0.070267	0.133900	0.223500
	POINT							
4	(682809.559 79530.149)	0.297900	0.189567	0.048200	0.078833	0.078367	0.132867	0.234000

5 rows × 31 columns



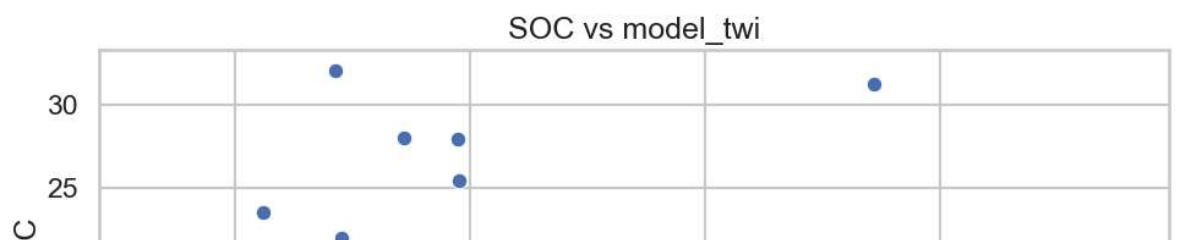
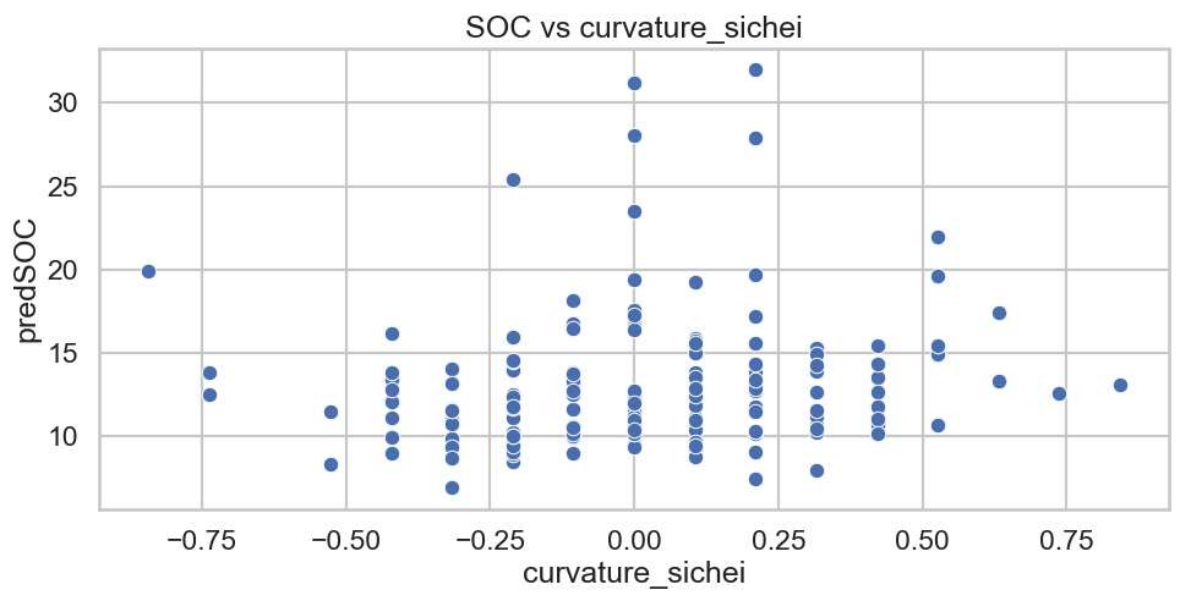
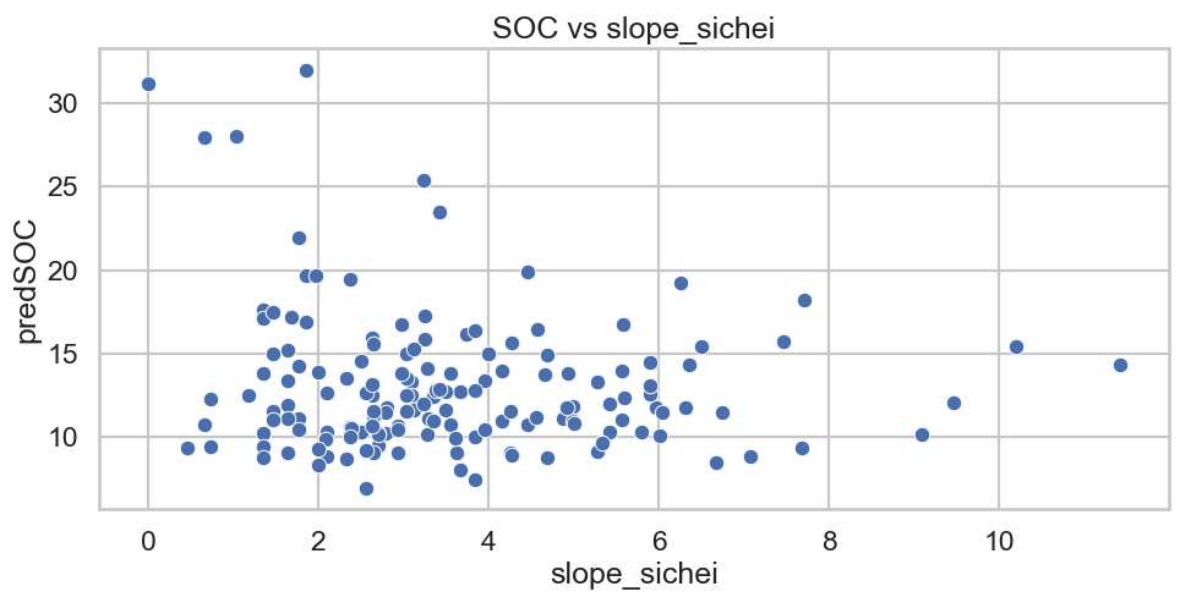
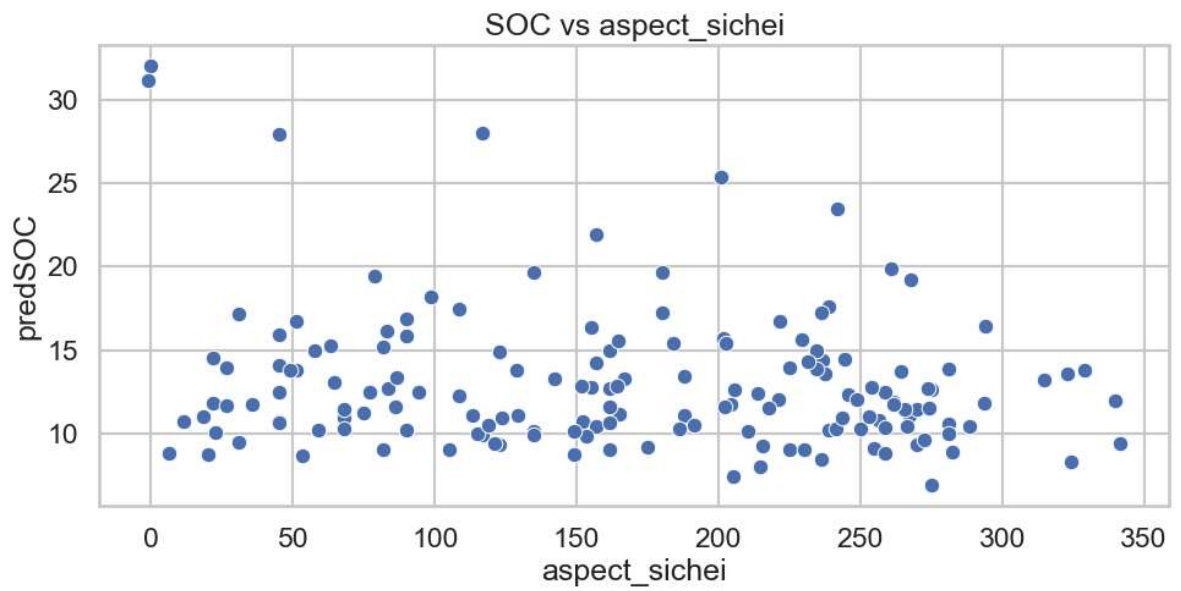
```
In [69]: print(soc_utm.isnull().sum())
        point_data = soc_utm.dropna() # For now, let's drop rows with any NaN values
```

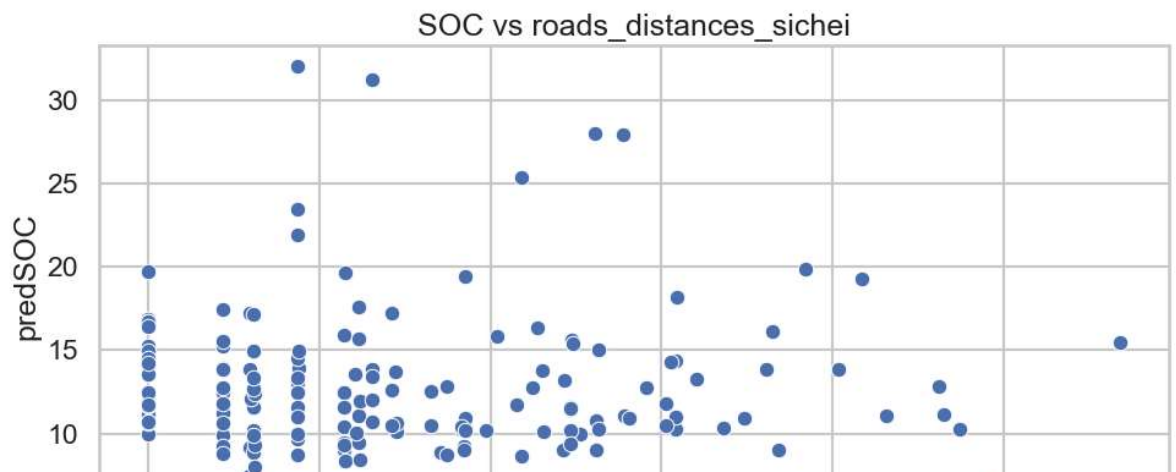
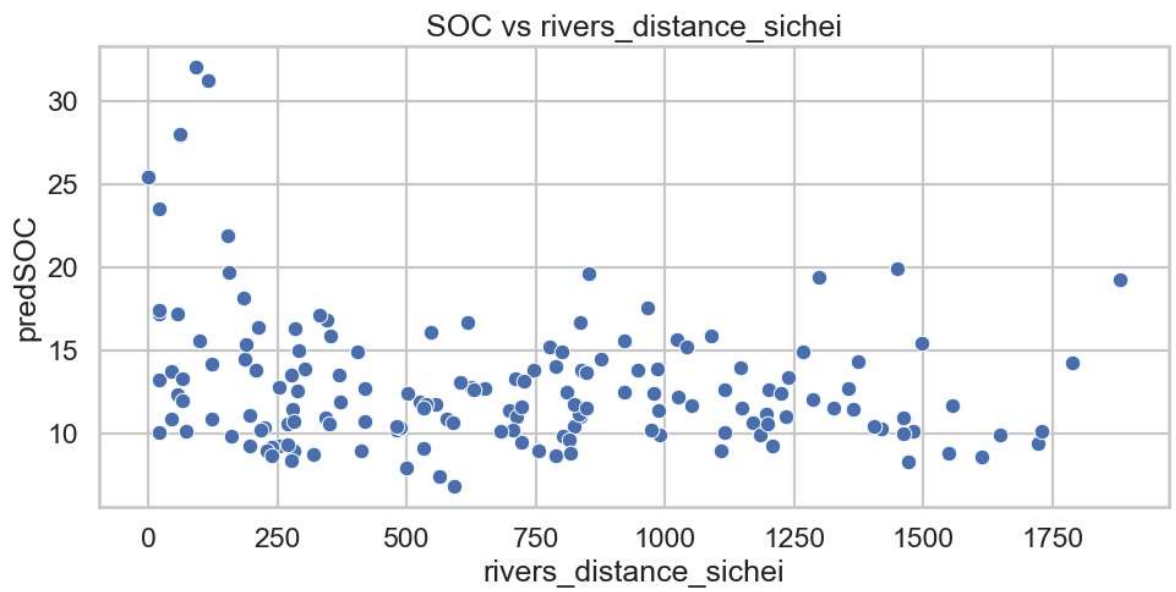
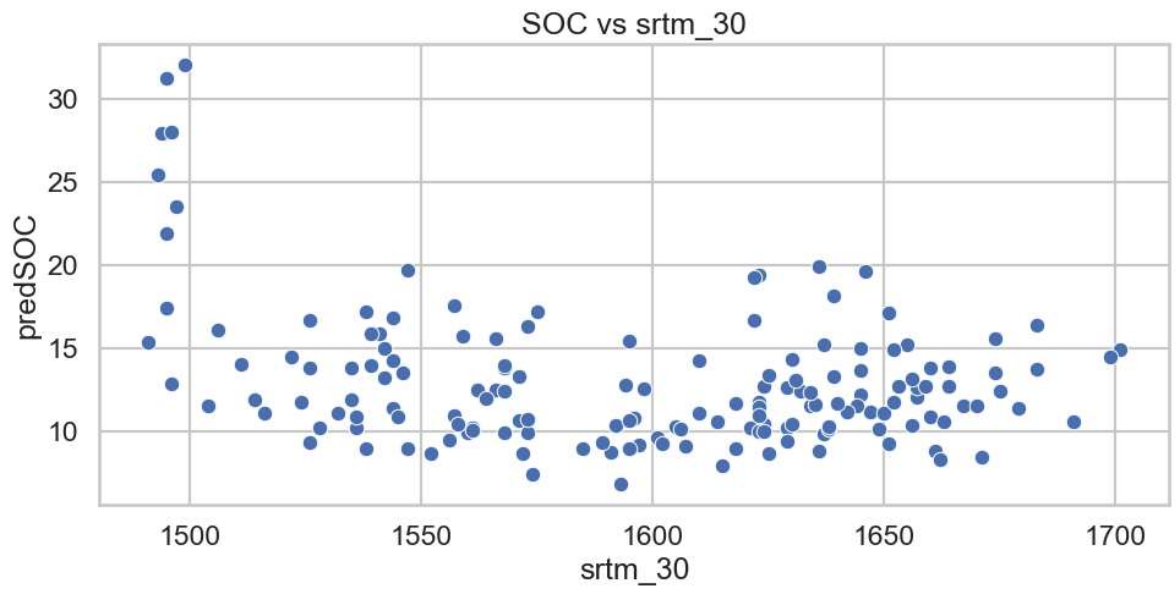
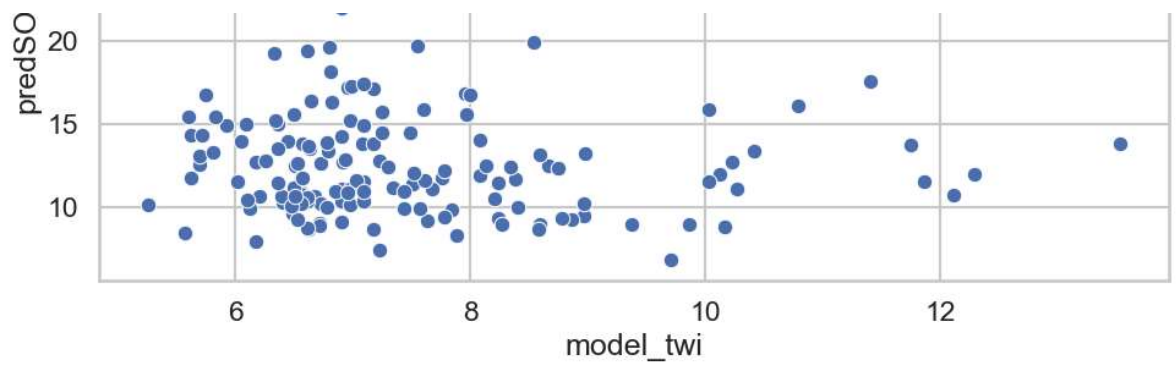
geometry	0
B11	0
B12	0
B2	0
B3	0
B4	0
B5	0
B6	0
B7	0
B8	0
B8A	0
B9	0
BSI	0
CPR	0
NBR	0
NDI	0
NDSI	0
NDVI	0
RVI	0
SAVI	0
VH	0
VV	0
VVHR	0
predSOC	0
aspect_sichei	0
slope_sichei	0
curvature_sichei	0
model_twi	0
srtm_30	0
rivers_distance_sichei	0
roads_distances_sichei	0

dtype: int64

Distribution Visualization

```
In [74]: fig, axes = plt.subplots(nrows=len(raster_files), ncols=1, figsize=(10, 5 * len(
    for i, column in enumerate(raster_files):
        get_first_name = lambda path: os.path.splitext(os.path.basename(path))[0]
        col_name = get_first_name(column)
        sns.scatterplot(x=point_data[col_name], y=point_data['predSOC'], ax=axes[i])
        axes[i].set_title(f'SOC vs {col_name}')
plt.tight_layout()
plt.show()
```

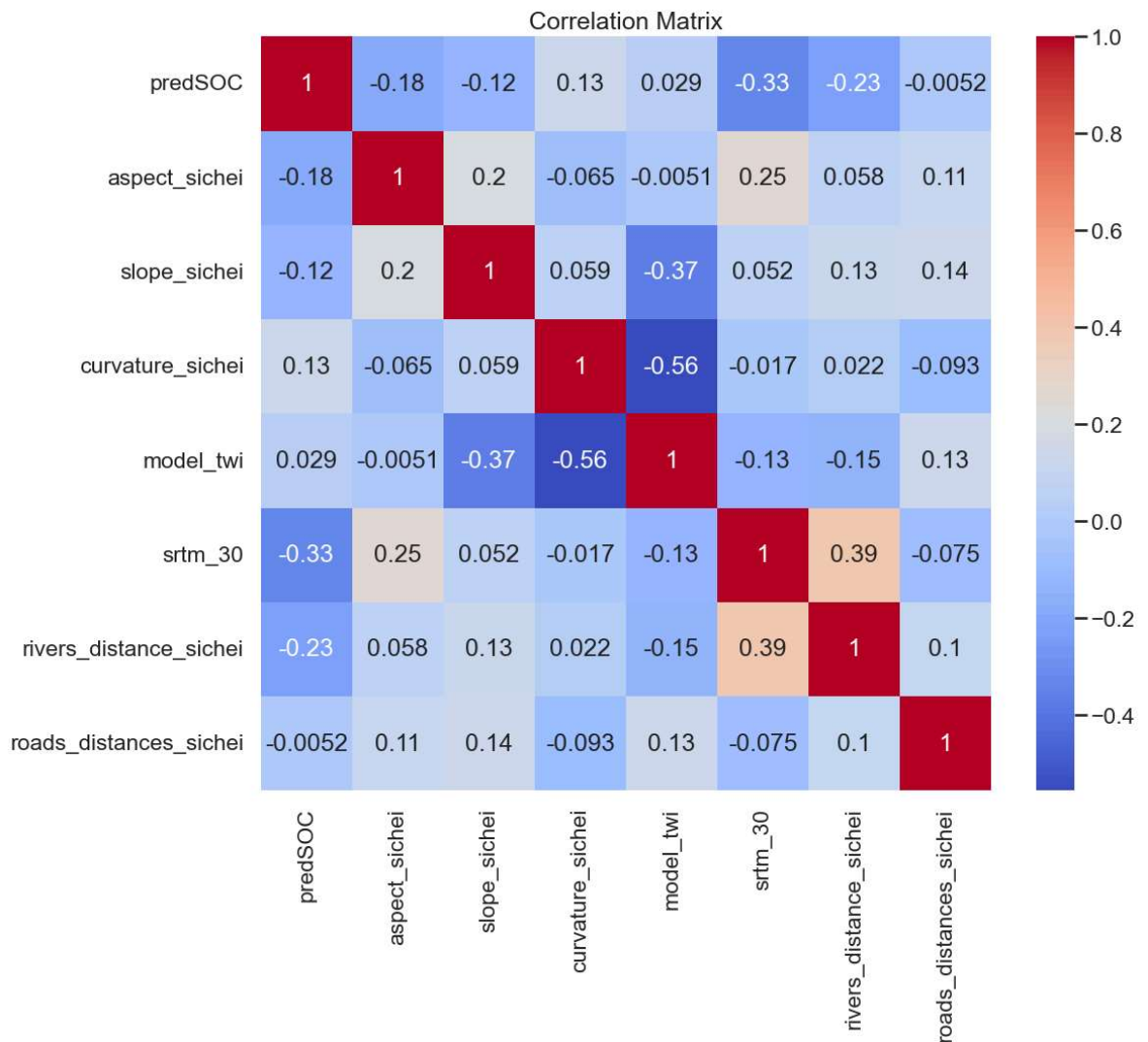






Correlation

```
In [75]: correlation_matrix = point_data[['predSOC'] + [get_first_name(col) for col in raster_files]]
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```



Modeling

```
In [82]: X = point_data[[get_first_name(col) for col in raster_files]]
y = point_data['predSOC'] # Replace 'your_soc_column'
```

```
In [83]: from sklearn.ensemble import RandomForestRegressor
from sklearn.feature_selection import RFE
from sklearn.model_selection import train_test_split

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

```

# RFE with Random Forest
rf_model = RandomForestRegressor(random_state=42)
rfe_rf = RFE(estimator=rf_model, n_features_to_select=16) # Adjust the number of
rfe_rf.fit(X_train, y_train)
print("RFE (Random Forest) Selected Features:", list(X_train.columns[rfe_rf.support_])

# RFE with XGBoost
from xgboost import XGBRegressor
xgb_model = XGBRegressor(random_state=42)
rfe_xgb = RFE(estimator=xgb_model, n_features_to_select=16) # Adjust the number
rfe_xgb.fit(X_train, y_train)
print("RFE (XGBoost) Selected Features:", list(X_train.columns[rfe_xgb.support_]

```

RFE (Random Forest) Selected Features: ['aspect_sichei', 'slope_sichei', 'curvature_sichei', 'model_twi', 'srtm_30', 'rivers_distance_sichei', 'roads_distances_sichei']

RFE (XGBoost) Selected Features: ['aspect_sichei', 'slope_sichei', 'curvature_sichei', 'model_twi', 'srtm_30', 'rivers_distance_sichei', 'roads_distances_sichei']

```

In [84]: from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_squared_error, r2_score

# Define parameter grids for Random Forest
param_grid_rf = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 3, 5]
}

# Grid Search for Random Forest
grid_search_rf = GridSearchCV(estimator=RandomForestRegressor(random_state=42),
                              param_grid=param_grid_rf,
                              cv=5, # 5-fold cross-validation
                              scoring='neg_mean_squared_error',
                              n_jobs=-1)
grid_search_rf.fit(X_train[X_train.columns[rfe_rf.support_]], y_train) # Use selected features
best_rf_model = grid_search_rf.best_estimator_
print("Best Random Forest Model:", best_rf_model)

# Define parameter grids for XGBoost
param_grid_xgb = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}

# Grid Search for XGBoost
grid_search_xgb = GridSearchCV(estimator=XGBRegressor(random_state=42),
                               param_grid=param_grid_xgb,
                               cv=5,
                               scoring='neg_mean_squared_error',
                               n_jobs=-1)
grid_search_xgb.fit(X_train[X_train.columns[rfe_xgb.support_]], y_train) # Use selected features
best_xgb_model = grid_search_xgb.best_estimator_
print("Best XGBoost Model:", best_xgb_model)

```

Best Random Forest Model: RandomForestRegressor(max_depth=5, n_estimators=300, random_state=42)

Best XGBoost Model: XGBRegressor(base_score=None, booster=None, callbacks=None, colsample_bylevel=None, colsample_bynode=None, colsample_bytree=1.0, device=None, early_stopping_rounds=None, enable_categorical=False, eval_metric=None, feature_types=None, feature_weights=None, gamma=None, grow_policy=None, importance_type=None, interaction_constraints=None, learning_rate=0.1, max_bin=None, max_cat_threshold=None, max_cat_to_onehot=None, max_delta_step=None, max_depth=3, max_leaves=None, min_child_weight=None, missing=nan, monotone_constraints=None, multi_strategy=None, n_estimators=200, n_jobs=None, num_parallel_tree=None, ...)

```
In [85]: # Evaluate Random Forest
y_pred_rf = best_rf_model.predict(X_test[X_test.columns[rfe_rf.support_]])
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2_rf = r2_score(y_test, y_pred_rf)
print(f"Random Forest RMSE on Test Set: {rmse_rf:.2f}")
print(f"Random Forest R-squared on Test Set: {r2_rf:.2f}")

# Evaluate XGBoost
y_pred_xgb = best_xgb_model.predict(X_test[X_test.columns[rfe_xgb.support_]])
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))
r2_xgb = r2_score(y_test, y_pred_xgb)
print(f"XGBoost RMSE on Test Set: {rmse_xgb:.2f}")
print(f"XGBoost R-squared on Test Set: {r2_xgb:.2f}")
```

Random Forest RMSE on Test Set: 3.35

Random Forest R-squared on Test Set: 0.48

XGBoost RMSE on Test Set: 3.40

XGBoost R-squared on Test Set: 0.47

```
In [86]: def predict_soc_raster(model, raster_files, output_path, profile, selected_features):
# Read all raster data into a stack
raster_data = []
for file in raster_files:
    with rasterio.open(file) as src:
        raster_data.append(src.read(1))

# Get the dimensions
num_rows, num_cols = raster_data[0].shape

# Initialize an empty array for predictions
predictions = np.zeros((num_rows, num_cols))

# Iterate through each cell
for i in range(num_rows):
    for j in range(num_cols):
        cell_values = [raster[i, j] for raster in raster_data]
        # Create a dictionary of feature values for the current cell
        cell_features = {raster_files[k].replace('.tif', ''): cell_values[k]}
        # Select only the features used in the model
        prediction_input = pd.DataFrame([cell_features])[selected_features]
        predictions[i, j] = model.predict(prediction_input)[0]

# Write the predictions to a new raster file
with rasterio.open(output_path, 'w', **profile) as dst:
    dst.write(predictions, 1)
```



```
# Get the profile from one of your input rasters
with rasterio.open(raster_files[0]) as src:
    profile = src.profile

output_prediction_path = './exports/predicted_soc.tif'
# Assuming best_rf_model and the features selected by RFE with RF
predict_soc_raster(best_rf_model, raster_files, output_prediction_path, profile,
print(f"Soil organic carbon prediction raster saved to: {output_prediction_path}")

# You would do a similar process if XGBoost was your chosen model
```



```

-----
KeyError                                Traceback (most recent call last)
Cell In[86], line 34
    32 output_prediction_path = './exports/predicted_soc.tif'
    33 # Assuming best_rf_model and the features selected by RFE with RF
--> 34 predict_soc_raster(best_rf_model, raster_files, output_prediction_path, p
rofile, list(X_train.columns[rfe_rf.support_]))
    35 print(f"Soil organic carbon prediction raster saved to: {output_predictio
n_path}")
    37 # You would do a similar process if XGBoost was your chosen model

Cell In[86], line 21, in predict_soc_raster(model, raster_files, output_path, pro
file, selected_features)
    19     cell_features = {raster_files[k].replace('.tif', ''): cell_values
[k] for k in range(len(raster_files))}
    20     # Select only the features used in the model
--> 21     prediction_input = pd.DataFrame([cell_features])[selected_feature
s]
    22     predictions[i, j] = model.predict(prediction_input)[0]
    24 # Write the predictions to a new raster file

File c:\Users\roywaswa\AppData\Roaming\mamba\envs\dsa\Lib\site-packages\pandas\co
re\frame.py:4108, in DataFrame.__getitem__(self, key)
    4106     if is_iterator(key):
    4107         key = list(key)
-> 4108     indexer = self.columns._get_indexer_strict(key, "columns")[1]
    4110 # take() does not accept boolean indexers
    4111 if getattr(indexer, "dtype", None) == bool:

File c:\Users\roywaswa\AppData\Roaming\mamba\envs\dsa\Lib\site-packages\pandas\co
re\indexes\base.py:6200, in Index._get_indexer_strict(self, key, axis_name)
    6197 else:
    6198     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
-> 6200 self._raise_if_missing(keyarr, indexer, axis_name)
    6202 keyarr = self.take(indexer)
    6203 if isinstance(key, Index):
    6204     # GH 42790 - Preserve name from an Index

File c:\Users\roywaswa\AppData\Roaming\mamba\envs\dsa\Lib\site-packages\pandas\co
re\indexes\base.py:6249, in Index._raise_if_missing(self, key, indexer, axis_nam
e)
    6247 if nmissing:
    6248     if nmissing == len(indexer):
-> 6249         raise KeyError(f"None of [{key}] are in the [{axis_name}]")
    6251 not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].unique
())
    6252     raise KeyError(f"{not_found} not in index")

KeyError: "None of [Index(['aspect_sichei', 'slope_sichei', 'curvature_sichei',
'srtm_30', 'rivers_distance_sichei', 'roads_distances_sichei'],\n
      dtype='object')] are in the [columns]"

```